

Assignment 4

ENME619 L01 - Machine Analysis

Josmar Cristello
UCID: 30158326

Table of Contents

1.	Introduction	3
2.	Questions	3
3.	Example #1 (Book's Page 131)	5
4.	Example #2 (Book's Page 160)	9
5.	External References	14
6.	Appendix: Matlab Code	14

Table of Figures

Figure 1. Shaping Process Setup (Book's Figure 4.3)	3
Figure 2. The table with modal parameters for Example #2 (Book's table 4.3) ^[1]	4
Figure 3. Transfer function obtained for Example #1.	5
Figure 4. Real part of the transfer function with an intersect in $y=0$. Points highlighted are the ones used for the chatter frequency (ω_c).	6
Figure 5. Stability lobes for Example #1 (2-DOF system)	8
Figure 6. Transfer functions for Example #2 (H_{xx}).	10
Figure 7. Transfer functions for Example #2 (H_{yy}).	10
Figure 8. Stability lobes for Example #2's slotting operation.	13
Figure 9. Stability lobes for Example #2's half immersion down milling operation.	13

1. Introduction

The goal of this report is to provide the answer and the calculations for the example of page 131 and page 160 of the 4th chapter of the book Manufacturing Automation, by Yusuf Altintas^[1]. The questions are also transposed from the book in their entirety in the following section. Section 2 contains the transcription of the questions, together with necessary material. Section 3 contains the solution for Example #1, and Section 4, for Example #2. Finally, section 5 and 6 are responsible for the references and the appendix (Matlab code).

2. Questions

The 1st example^[1] poses the following:

Example #1 (Book's Page 131). Stability of a Shaping System with Two Degrees of Freedom. A shaping process with a two-degree-of-freedom (2-DOF) system is shown in Figure 4.3. The system flexibilities in directions x_1 and x_2 are given as:

$$x_1 \leftarrow \omega_{n1} = 250 \text{ Hz}, \zeta_1 = 1.2\%, k_1 = 2.26 \times 10^8 \text{ N/m}$$

$$x_2 \leftarrow \omega_{n2} = 150 \text{ Hz}, \zeta_2 = 1.0\%, k_2 = 2.13 \times 10^8 \text{ N/m}$$

The cutting force is $F = K_f \cdot a \cdot h(t)$, where the cutting constant $K_f = 1,000 \text{ MPa}$. The flexibilities are oriented with $\theta_1 = 30^\circ$ and $\theta_2 = 45^\circ$ from the y axis. Plot the stability lobe of the system.

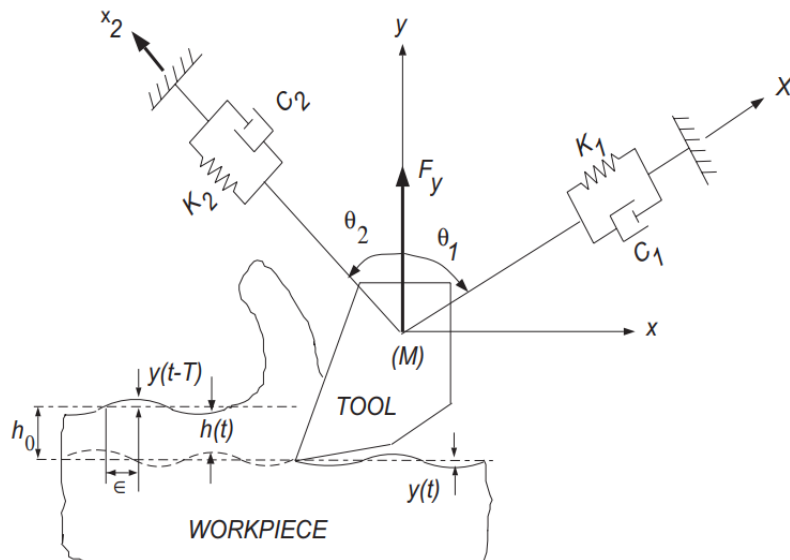


Figure 1. Shaping Process Setup (Book's Figure 4.3).

The 2nd example^[1] poses the following:

Example #2 (Book's Page 160) End Milling with a Flexible Cutter. A helical carbide end mill with four flutes, 19.05 mm diameter, and 10 degree rake is used in peripheral milling of aluminum alloy Al-7075. The identified cutting constants for the work material and cutter are found as $K_t = 796 \text{ [N/mm}^2\text{]}$ and $K_r = 0.212$. The transfer function of the cutter attached to the spindle is measured at the tool tip in both

feed (x) and normal (y) directions with an instrumented hammer and accelerometer. The modal parameters of the structure converted to displacement domain are identified from modal analysis software and are given in Table 4.3. The experimentally measured and reconstructed transfer functions are compared in Figure 4.16, which indicates the accuracy of identified modal parameters. Later the modal parameters are used to simulate the stability lobes for a half-immersion down-milling and slotting of Al-7075 alloy. These results are shown in Figure 4.17. The minimum stable depth of cut seems to be about $a_{lim} = 4$ mm in half-immersion up-milling and about $a_{lim} = 1.5$ mm in slotting. The ideal stability pockets correspond to a spindle speed $n = 11,800$ rev/min, where the maximum material can be removed without chatter. Note that there is hardly any stability pocket at speeds less than $n = 5,000$ rev/min, which is due to the many tightly packed vibration waves at each tooth period. Because Al-7075 is not a hard material, it is clearly advisable to machine it at high speeds to enhance productivity. The stability lobes presented for this case have been experimentally proven on a machining center powered by a 15 kW spindle capable of 15,000 rev/min.

The table referenced by the text (Table 4.3) is also transposed in its entirety in the following figure.

TABLE 4.3. Identified Modal Parameters of a Vertical Machining Center in x and y Directions. The transfer function units are in [m/N]			
Mode x	ω_{nx} [Hz]	ζ_x	$\sigma_x + j\nu_x$
1	262.16	0.048	1.994606E-06-j 3.779091E-06
2	503.60	0.096	1.932445E-06-j 2.44616E-05
3	667.92	0.057	1.364547E-05-j 4.128093E-05
4	886.78	0.074	8.632813E-07-j 7.195716E-05
5	2,201.5	0.015	-5.011467E-06-j 4.623555E-05
6	2,799.4	0.027	-1.394105E-05-j 8.317728E-05
7	3,837.7	0.007	-6.874543E-06-j 6.560925E-05
8	4,923.6	0.019	-8.824935E-06-j 6.54481E-05
9	6,038.7	0.025	2.564520E-06-j 1.986276E-05
Mode y	ω_{ny} [Hz]	ζ_y	$\sigma_y + j\nu_y$
1	285.53	0.021	9.797268E-07-j 5.959362E-07
2	587.8	0.089	1.349004E-05-j 4.544067E-06
3	749.6	0.027	-3.106831E-05-j 3.816475E-05
4	804.9	0.075	3.033849E-05-j 8.813244E-05
5	1,573.7	0.027	1.283945E-06-j 8.678961E-06
6	2,038.1	0.016	1.298625E-06-j 1.270846E-05
7	2,303.3	0.220	9.518897E-07-j 3.750897E-05
8	2,681.0	0.019	1.190834E-05-j 2.825781E-05
9	2,870.5	0.014	-1.912932E-05-j 4.051860E-05
10	3,838.8	0.006	-1.119589E-05-j 8.475725E-05
11	4,928.9	0.017	-1.859880E-05-j 8.993226E-05
12	6,073.4	0.016	-7.608392E-06-j 2.022991E-05

Figure 2. The table with modal parameters for Example #2 (Book's table 4.3)^[1].
(Mode x = feed direction, Mode y = Normal direction)

3. Example #1 (Book's Page 131)

To obtain the stability lobes of the system, as the question poses, the first step is to obtain the transfer function that describe the system in the y-direction. This system, in particular, is being modelled with two degrees of freedom (DOFs), as per **Figure 1**, and thus the contribution of each DOF will be considered individually. The transfer function can then be represented with:

$$\Phi_y(s) = \frac{Y(s)}{F(s)} = \frac{\omega_{n1}^2 \cos^2(\theta_1)}{k_1 \cdot (s^2 + 2\zeta_1 \omega_{n1} s + \omega_{n1}^2)} + \frac{\omega_{n2}^2 \cos^2(\theta_2)}{k_2 \cdot (s^2 + 2\zeta_2 \omega_{n2} s + \omega_{n2}^2)} \quad \text{Eq 3.1}$$

Where (from the problem statement):

- $\omega_{n1} = 250 \text{ Hz} \cdot 2\pi \cong 1570.8 \text{ rad/s.}$
- $\omega_{n2} = 150 \text{ Hz} \cdot 2\pi \cong 942.5 \text{ rad/s.}$
- $\zeta_1 = 1.2 \text{ \%}.$
- $\zeta_2 = 1.0 \text{ \%}.$
- $\theta_1 = 30^\circ \cdot 2\pi/360 = 0.524 \text{ rad.}$
- $\theta_2 = 30^\circ \cdot 2\pi/360 = -0.785 \text{ rad.}$
- $k_1 = 2.26 \cdot 10^8 \text{ N/m.}$
- $k_2 = 2.13 \cdot 10^8 \text{ N/m.}$

Then, with the transfer function, it is trivial to obtain the magnitude & phase plots and the real & imaginary parts. Operationally, the magnitude & phase are obtained with the *bode*^[2] function from MATLAB and real & imaginary parts with the *nyquist*^[3] function. Those all shown in **Figure 3**.

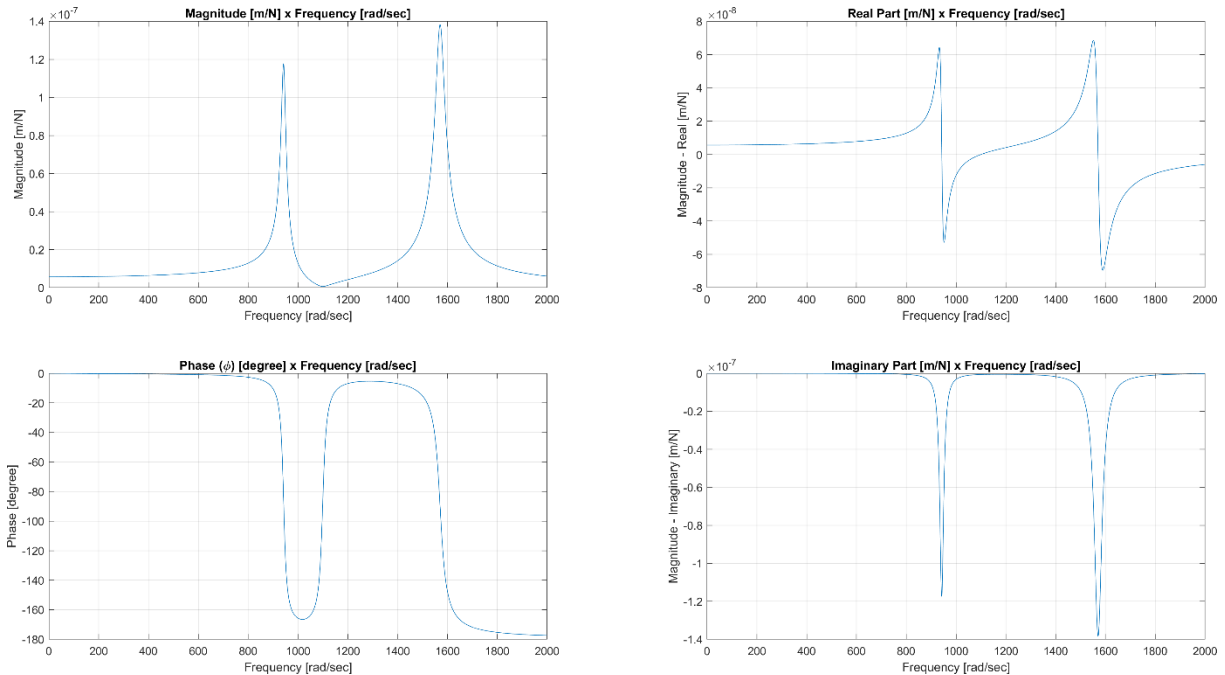


Figure 3. Transfer function obtained for Example #1.

The most important part of the obtaining the transfer function, however, is determining the chatter frequency. The plot with the real part (i.e. top-right of Figure 3) is particularly useful for that. If the solution of the real part is positive, the solution will have an exponential term, and the chatter vibrations will grow indefinitely. On the other hand, if it is negative, the vibrations will be suppressed with time and the system stabilizes. When it is exactly zero, the workpiece oscillates with a constant amplitude at chatter frequency (ω_c). The process of identifying those points is illustrated in Figure 4, and the frequencies are:

- 150.1, 171.7 Hz (942.9, 1078.6 rad/s) around ω_{n1} ;
- 250.0, 318.3 Hz (1570.7, 2000.0 rad/s) around ω_{n2} .
-

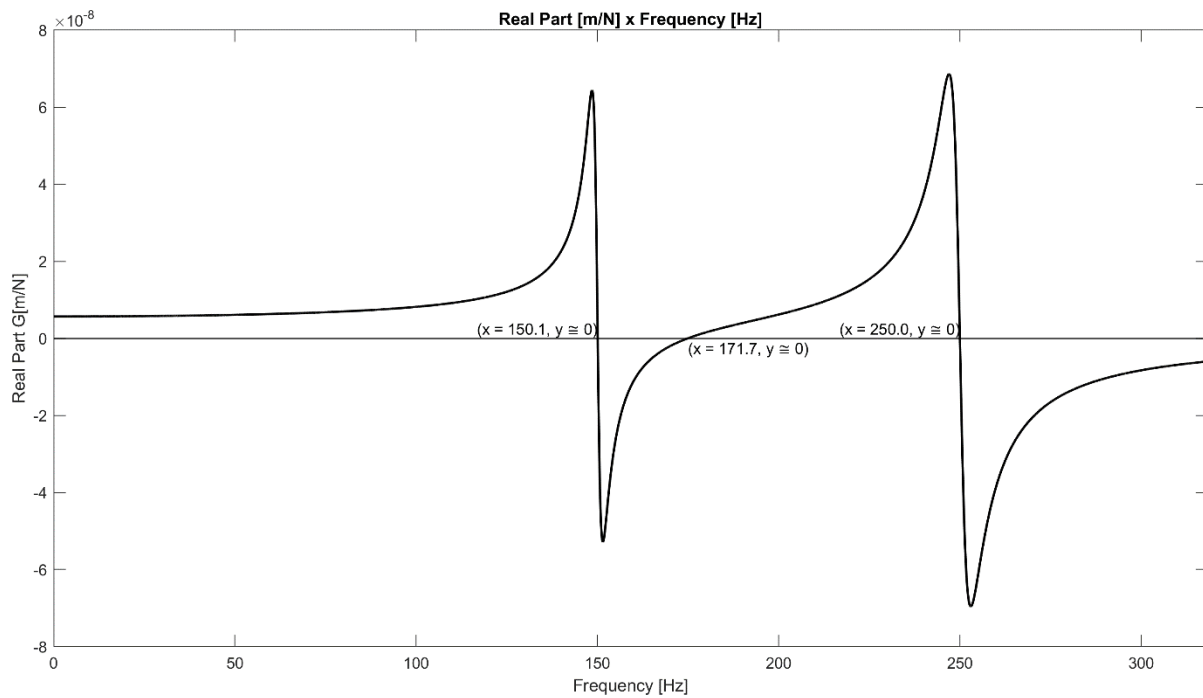


Figure 4. Real part of the transfer function with an intersect in $y=0$. Points highlighted are the ones used for the chatter frequency (ω_c).

If we then consider that we are working on critical borderline stability (when $s = j \cdot \omega_c$), the characteristic function becomes:

$$1 + (1 - e^{-j\omega_c T})K_f \cdot a_{lim} \Phi(j \cdot \omega_c) = 0 \quad \text{Eq 3.2}$$

Where a_{lim} is a new and useful concept introduced for chatter analysis: The maximum axial depth of cut without chatter vibrations. We can then separate the transfer function in real and imaginary parts the same way as done before, and calculate the phase shift (ψ) of the structure:

$$\tan(\psi) = \frac{H(\omega_c)}{G(\omega_c)} \quad \text{Eq 3.3}$$

Note: Operationally, this is done with the function $\text{atan2}^{[4]}$, from MATLAB.

The maximum axial depth of cut (a_{lim}) can also be computed:

$$a_{lim} = \frac{-1}{2K_f G(\omega_c)} \quad \text{Eq 3.4}$$

Finally, the spindle speed (n) and period (T) for each stability lobe is calculated, with:

$$T = \frac{2k\pi + \epsilon}{2\pi f_c} \quad \text{Eq 3.5}$$

$$n = \frac{60}{T} \quad \text{Eq 3.6}$$

Where:

- $k \rightarrow$ Stability lobe index
- $\epsilon = 3\pi + 2\psi \rightarrow$ Phase shift between inner and outer waves
- $f_c \rightarrow$ Chatter Frequency [Hz]

Now that the main equations and variables are defined, the whole procedure can be outlined:

- **Step 1.** Select a chatter frequency (ω_c) in the negative region of the real part of the transfer function (Illustrated in Figure 4)
- **Step 2.** Calculate the phase shift of the structure (ψ) with this chatter frequency (ω_c), as per Eq 3.3.
- **Step 3.** Calculate the maximum axial depth of cut (a_{lim}), as per Eq 3.4.
- **Step 4.** Calculate the spindle speed (N) and period (T) for each stability lobe (k), as per Eq 3.5 and Eq 3.6.
- **Step 5.** Repeat steps 1-4 for other chatter frequencies around the natural frequencies (ω_{n1} and ω_{n2})

This procedure is then applied, and the results can be seen in Figure 1. It is particularly interesting to note that there is a significant pocket around the 7000 rev/min spindle speed range, which would allow for a 60 mm depth of cut. This would be useful in a real-life scenario where the goal is to maximize productivity.

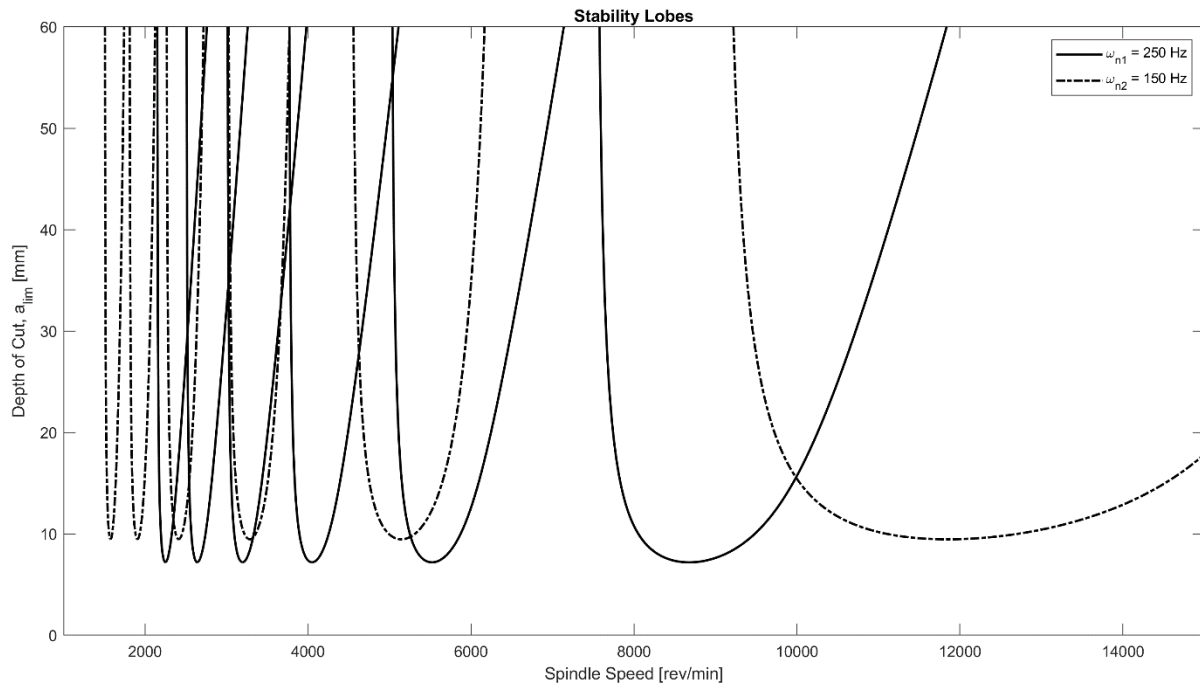


Figure 5. Stability lobes for Example #1 (2-DOF system).

4. Example #2 (Book's Page 160)

Somewhat similarly to Example #1, the first step is to define the transfer function of the system. However, in this case, Figure 2 outlines very extensive data that simplifies the process of obtaining those transfer functions. Another crucial different from the previous example, is that this one is a multiple degree of freedom system (MDOF), whereas the previous was a 2-DOF system. To obtain the transfer functions, first, the residuals are going to be calculated:

$$R = \alpha + \beta s \quad \text{Eq 4.1}$$

$$\alpha = 2 \cdot (\zeta \omega_n \sigma - \omega_d \nu) \quad \text{Eq 4.2}$$

$$\beta = 2 \cdot \sigma \quad \text{Eq 4.3}$$

Where:

- $\omega_d = \omega_n \sqrt{1 - \zeta^2} \rightarrow$ Damped natural frequency.
- $\sigma \Rightarrow$ Phase shift between inner and outer waves.
- $\nu \Rightarrow$ Chatter Frequency [Hz]

Note that this is done, as per Figure 2, for both the x direction (feed) and the y direction (normal). With the residuals, the transfer functions ($H_{xx}(S)$ and $H_{yy}(S)$) are trivially obtained with the following equations:

$$H_{xx}(S) = \sum_{k=1}^9 \frac{[R]_k}{s^2 + 2\zeta_{nk}\omega_{nk}s + \omega_{nk}^2} \quad \text{Eq 4.4}$$

$$H_{yy}(S) = \sum_{k=1}^{12} \frac{[R]_k}{s^2 + 2\zeta_{nk}\omega_{nk}s + \omega_{nk}^2} \quad \text{Eq 4.5}$$

Where k is the mode number, as per the experimental data in Figure 2. Once this procedure is completed, the transfer functions are shown in Figure 6 and Figure 7. Note that they are plotted until $7,000 \text{ Hz} \cdot 2\pi = 43,982 \text{ rad/s}$, to match the same visualization range of the textbook^[1].

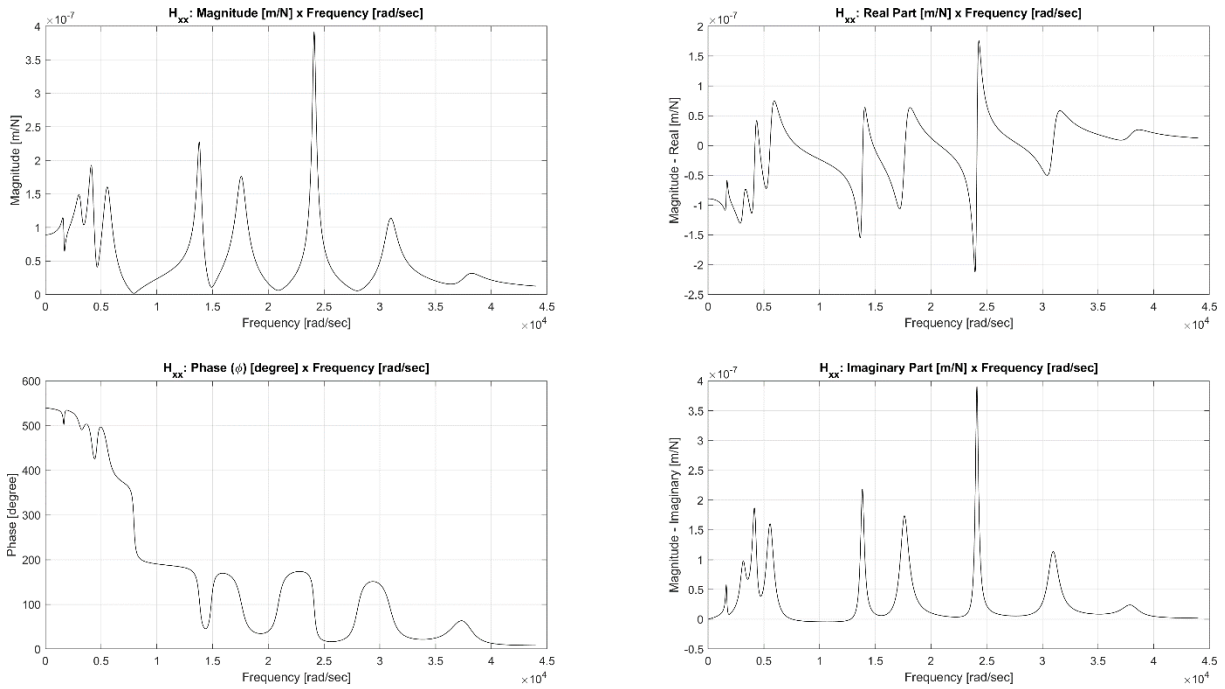


Figure 6. Transfer functions for Example #2 (H_{xx}).

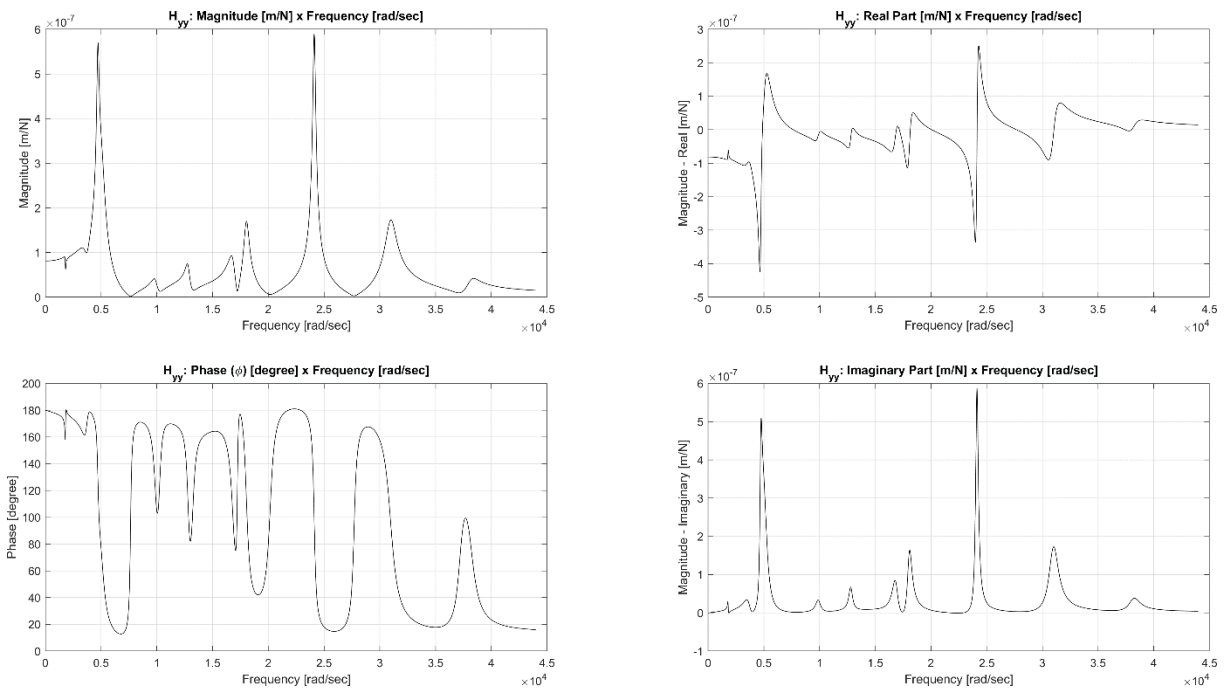


Figure 7. Transfer functions for Example #2 (H_{yy}).

With the transfer functions we can proceed to the zero-order solution of chatter stability. A note here, that we are considering the average Fourier coefficient (A_0) with $r=0$. With this, we have the following directional matrix:

$$[A_0] = \frac{1}{T} \int_{\phi_{st}}^{\phi_{ex}} [A(\phi)] d\phi = \frac{N}{2\pi} \begin{bmatrix} \alpha_{xx} & \alpha_{xy} \\ \alpha_{yx} & \alpha_{yy} \end{bmatrix} \quad \text{Eq 4.6}$$

Where each direction dynamic milling coefficient is an integrated function (α) that can be calculated with:

$$\alpha_{xx} = \frac{1}{2} [\cos(2\phi) - 2K_r\phi + K_r \sin(2\phi)]_{\phi_{st}}^{\phi_{ex}} \quad \text{Eq 4.7}$$

$$\alpha_{xy} = \frac{1}{2} [-\sin(2\phi) - 2\phi + K_r \cos(2\phi)]_{\phi_{st}}^{\phi_{ex}} \quad \text{Eq 4.8}$$

$$\alpha_{yx} = \frac{1}{2} [-\sin(2\phi) + 2\phi + K_r \cos(2\phi)]_{\phi_{st}}^{\phi_{ex}} \quad \text{Eq 4.9}$$

$$\alpha_{yy} = \frac{1}{2} [-\cos(2\phi) - 2K_r\phi - K_r \sin(2\phi)]_{\phi_{st}}^{\phi_{ex}} \quad \text{Eq 4.10}$$

This system has a characteristic equation with the following eigenvalues:

$$\Lambda = -\frac{N}{4\pi} a K_t (1 - e^{-i\omega_c T}) \quad \text{Eq 4.11}$$

And thus, the characteristic equation becomes:

$$\det[[I] + \Lambda[\Phi_0(i\omega_c)]] = 0 \quad \text{Eq 4.12}$$

After solving for the given values – chatter frequency (ω_c), cutting coefficient factors (K_t, K_r), radial immersion angles (ϕ_{st}, ϕ_{ex}), and considering the two directions (Φ_{xx}, Φ_{yy}) disconsidering the others ($\Phi_{xy} = \Phi_{yx} = 0$), we have a much simpler, quadratic characteristic equation:

$$a_0 \Lambda^2 + a_1 \Lambda + 1 = 0 \quad \text{Eq 4.13}$$

Where:

$$a_0 = \Phi_{xx}(i\omega_c) \Phi_{yy}(i\omega_c) (\alpha_{xx} \alpha_{yy} - \alpha_{xy} \alpha_{yx}) \quad \text{Eq 4.14}$$

$$a_1 = \alpha_{xx} \Phi_{xx}(i\omega_c) + \alpha_{yy} \Phi_{yy}(i\omega_c) \quad \text{Eq 4.15}$$

Finally, the depth of cut can be calculated with:

$$a_{lim} = -\frac{2\pi}{NK_t} \left[\frac{\Lambda_R(1 - \cos(\omega_c T)) + \Lambda_I \sin(\omega_c T)}{1 - \cos(\omega_c T)} \right] + i \left[\frac{\Lambda_I(1 - \cos(\omega_c T)) + \Lambda_R \sin(\omega_c T)}{1 - \cos(\omega_c T)} \right] \quad \text{Eq 4.16}$$

Depth of cut, however, is a real number. Because of this, the imaginary part vanishes, and we are left with:

$$a_{lim} = -\frac{2\pi\Lambda_R}{NK_t}(1 + \kappa^2) \quad \text{Eq 4.17}$$

Where κ was introduced to simplify the equation, and simply stands for:

$$\kappa = \frac{\Lambda_I}{\Lambda_R} = \frac{\sin(\omega_c T)}{1 - \cos(\omega_c T)} \quad \text{Eq 4.18}$$

The phase shift (ψ) is found with:

$$\tan \psi = \kappa = \frac{\Lambda_I}{\Lambda_R} \quad \text{Eq 4.19}$$

Finally, the spindle speed (n) and tooth-passing period (T) are calculated in a very similar manner to the previous example (i.e., Eqs 3.5 and 3.6):

$$T = \frac{1}{\omega_c}(\epsilon + 2k\pi) \quad \text{Eq 4.20}$$

$$n = \frac{60}{NT} \quad \text{Eq 4.21}$$

Now with the main equations and variables are defined, the whole procedure can be outlined:

- **Step 1.** Select a chatter frequency (ω_c) around a dominant mode (Similar to Figure 4). For this case, the selected frequencies were: 600, 3600 and 4600 Hz. All with a +300 Hz scanning range.
- **Step 2.** Solve the eigenvalue (Eq. 4.13)
- **Step 3.** Calculate the maximum axial depth of cut (a_{lim}), as per Eq 3.17.
- **Step 4.** Calculate the spindle speed (N) and period (T) for each stability lobe (k), as per Eq 4.18 and Eq 4.19.
- **Step 5.** Repeat steps 1-4 for other chatter frequencies around all dominant modes of the structure.

This procedure is then applied, and the results can be seen in Figure 8 for slotting, and on Figure 9 for half immersion down milling. It is interesting to confirm the intuition – for slotting, the maximum depth of cut is much lesser than for slotting. This makes sense, as the tool is engaging relatively less material per rotation.

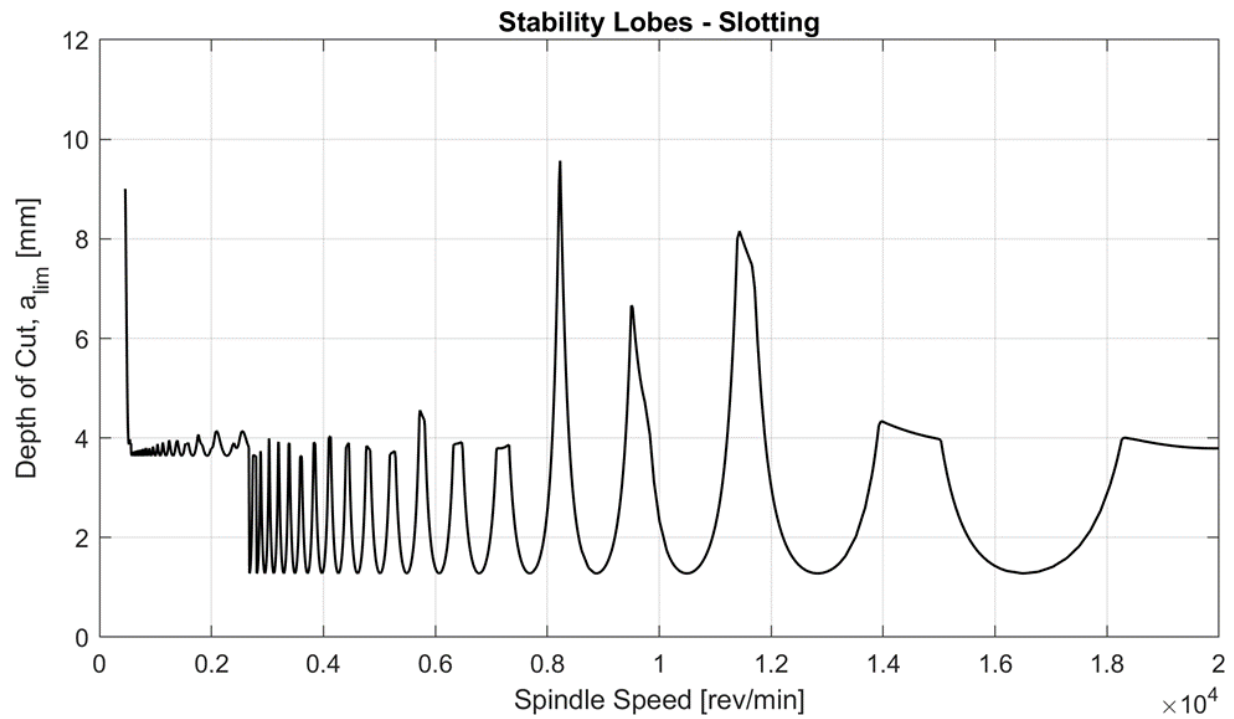


Figure 8. Stability lobes for Example #2's slotting operation.



Figure 9. Stability lobes for Example #2's half immersion down milling operation.

5. External References

1. Altintas, Y. (2012). *Manufacturing automation: Metal cutting mechanics, machine tool vibrations, and CNC design* (2nd ed). Cambridge University Press.
2. *Bode plot of frequency response, or magnitude and phase data—MATLAB bode.* (n.d.). Retrieved March 27, 2022, from <https://www.mathworks.com/help/control/ref/lti.bode.html>
3. *Nyquist plot of frequency response—MATLAB nyquist.* (n.d.). Retrieved March 27, 2022, from <https://www.mathworks.com/help/control/ref/lti.nyquist.html>
4. *Four-quadrant inverse tangent—MATLAB atan2.* (n.d.). Retrieved March 27, 2022, from <https://www.mathworks.com/help/matlab/ref/atan2.html>

6. Appendix: Matlab Code

For Example #1 (Book's Page 131):

```
% Josmar Cristello
% Assignment #4
% Example #1 (Book's Page 131)

clf; clear all; close all;

freq_n_1 = 250; % Hz
freq_n_2 = 150; % Hz

omega_n_1 = freq_n_1*2*pi(); % Hz → rad/s
omega_n_2 = freq_n_2*2*pi(); % Hz → rad/s

zeta_1 = 1.2/100; % unitless
zeta_2 = 1.0/100; % unitless

k_1 = 2.26*10^8; % N/m
k_2 = 2.13*10^8; % N/m

% Is it 10^6 or 10^9? → Confusing on the book text %
K_f = 1*10^6; % Pa, Cutting constant (Feed direction).

theta_1 = 30*(2*pi()/360); % Radians, Flexibilities
theta_2 = -45*(2*pi()/360); % Radians, Flexibilities

% Question: Plot stability lobe of the system
```

```

% ===== %
% Answer %
% ===== %

% Calculating the Transfer Function (y Direction)
transfer_func = tf(omega_n_1^2*cos(theta_1)^2,k_1*[1, 2*zeta_1*omega_n_1,
omega_n_1^2]) + ...
                tf(omega_n_2^2*cos(theta_2)^2,k_2*[1, 2*zeta_2*omega_n_2,
omega_n_2^2]);

% Transfer Function Plots: Magnitude & Phase
f=0:.1:2000;
[magH phaH]=bode(transfer_func,f);
magH = squeeze(magH);
phaH = squeeze(phaH);

subplot_tight(3,3,0+1,0.075); plot(f, magH);
title("Magnititude [m/N] x Frequency [rad/sec]"); xlabel('Frequency [rad/sec]');
ylabel('Magnititude [m/N]'); grid on;

subplot_tight(3,3,3+1,0.075); plot(f, phaH);
title("Phase (\phi) [degree] x Frequency [rad/sec]"); xlabel('Frequency
[rad/sec]'); ylabel('Phase [degree]'); grid on;

% Transfer Function Plots: Real & Imaginary
[Real Imag]=nyquist(transfer_func,f);
Real = squeeze(Real);
Imag = squeeze(Imag);

subplot_tight(3,3,0+2,0.075); plot(f, Real);
title('Real Part [m/N] x Frequency [rad/sec]'); xlabel('Frequency [rad/sec]');
ylabel('Magnititude - Real [m/N]'); grid on;

subplot_tight(3,3,3+2,0.075); plot(f, Imag)
title('Imaginary Part [m/N] x Frequency [rad/sec]'); xlabel('Frequency [rad/sec]');
ylabel('Magnititude - Imaginary [m/N]'); grid on;

my_export("A4_Q1_Transfer Functions.png");

% ===== %
% Finding Stability Lobes %
% ===== %

% Finding points where G() is negative
pt1 = (1570.7)/(2*pi);
pt2 = (2000)/(2*pi);
resolution = 5000;
division = (pt2 - pt1)/resolution;

for i = 1:resolution
    freq_c = pt1 + i*division;           % Hz, Chatter Frequency
    omega_c = freq_c*2*pi();             % Hz → rad/s, Chatter Frequency
    [G,H]=nyquist(transfer_func,omega_c);
    psi = atan2(H,G);

```

```

a_lim = -1/(2*K_f*G);
stability_matrix_omega_n_1(i,1) = a_lim;
for k = 0:7
    T = (2*(k)*pi() + (3*pi()+2*psi))/(2*pi()*freq_c);
    stability_matrix_omega_n_1(i,(k+2)) = 60/T;
end
end

size_matrix = size(stability_matrix_omega_n_1,2);
for i = 1:(size_matrix-2)
    tmp = sortrows([stability_matrix_omega_n_1(:,1),
stability_matrix_omega_n_1(:,size_matrix-i)],2);
    if i == 1
        stability_matrix_omega_n_1_sorted = tmp;
    else
        stability_matrix_omega_n_1_sorted = [stability_matrix_omega_n_1_sorted;
tmp];
    end
end

% Finding points where G() is negative
pt1 = (942.9)/(2*pi);
pt2 = (1078.6)/(2*pi);
division = (pt2 - pt1)/resolution;

for i = 1:resolution
    freq_c = pt1 + i*division;           % Hz, Chatter Frequency
    omega_c = freq_c*2*pi();             % Hz → rad/s, Chatter Frequency
    [G,H]=nyquist(transfer_func,omega_c);
    psi = atan2(H,G);
    a_lim = -1/(2*K_f*G);
    stability_matrix_omega_n_2(i,1) = a_lim;
    for k = 0:6 % Stability Lobes
        T = (2*(k)*pi() + (3*pi()+2*psi))/(2*pi()*freq_c);
        stability_matrix_omega_n_2(i,(k+2)) = 60/T;
    end
end

size_matrix = size(stability_matrix_omega_n_2,2);
for i = 1:(size_matrix-2)
    tmp = sortrows([stability_matrix_omega_n_2(:,1),stability_matrix_omega_n_2(:,
size_matrix-i)],2);
    if i == 1
        stability_matrix_omega_n_2_sorted = tmp;
    else
        stability_matrix_omega_n_2_sorted = [stability_matrix_omega_n_2_sorted;
tmp];
    end
end

figure;
subplot_tight(2,2,0+1,0.075); plot(f./(2*pi), Real, 'k', 'linewidth',1.5); %Wn2
stability plot
xlim([0 2000/(2*pi)])

```

```

title('Real Part [m/N] x Frequency [Hz]'); xlabel("Frequency [Hz]"); ylabel("Real
Part G[m/N]");
labelpoints(942.9/(2*pi), 0, "(x = 150.1, y  $\cong$  0)"); % Clever way to label the
points
labelpoints(1099.4/(2*pi), 0, "(x = 171.7, y  $\cong$  0)","SE"); % Clever way to label
the points
labelpoints(1570.7/(2*pi), 0, "(x = 250.0, y  $\cong$  0)"); % Clever way to label the
points
%hold on;
yline(0,'k', 'linewidth', 1.0);

% Plotting in the same color scheme as the book
subplot_tight(2,2,0+2,0.075); plot(stability_matrix_omega_n_1_sorted(:,2),
stability_matrix_omega_n_1_sorted(:,1), 'k', 'linewidth',1.5); %W_n2
hold on
plot(stability_matrix_omega_n_2_sorted(:,2),
stability_matrix_omega_n_2_sorted(:,1), 'k-.', 'linewidth',1.5);% W_n1
ylim([0 60])
xlim([1000 15000])
title('Stability Lobes'); xlabel('Spindle Speed [rev/min]'); ylabel('Depth of Cut,
a_{lim} [mm]')
legend('\omega_{n1} = 250 Hz', '\omega_{n2} = 150 Hz');
my_export("A4_Q1_Stability_Lobes");

function y = my_export(title)
    s = get(0, 'ScreenSize');
    set(gcf, 'Position', [0 0 s(3) s(4)]);
    set(gcf, 'Color', 'w');
    export_fig(title, "-png", "-r300");
end

```

For Example #2 (Book's Page 160):

```

% Josmar Cristello
% Assignment #4
% Example #2 (Book's Page 160)

clf; clear all; close all;

% Data from Question %
D = 19.05*10^-3; % m, Diameter
N = 4; % #, Number of flutes
K_t = 796; % N/mm^2 Tangential cutting Constant (N/m^2)
K_t = K_t*10^3; % N/mm^2  $\rightarrow$  N/m^2
K_r = 0.212; % Unitless, Radial cutting constant.

phi_ex = 180*2*pi()/360; % radians
phi_st = 0*2*pi()/360; % radians

N_max = 15000; %rev/min, max speed spindle is capable of

% Data from Table 4.3 %

```

```

% Mode x %
freq_n_x = [262.16,503.60,667.92,886.78,2201.5,2799.4,3837.7,4923.6,6038.7]; % Hz
omega_n_x = freq_n_x.*2*pi(); % Hz
→ rad/s, nat freq for mode x (Feed Direction)
zeta_x = [0.048,0.096,0.057,0.074, 0.015, 0.027,0.007,0.019,0.025];
sig_x = [1.994606,1.932445, 13.64547, 0.8632813,-5.011467,-13.94105,-6.874543,-8.824935,2.564520].*10^-6;
nu_x = [0.3779091, 2.44616, 4.128093, 7.195716, 4.623555, 8.317728, 6.560925, 6.54481, 1.986276].*10^-5;

% Mode y %
freq_n_y = [285.53,587.8,749.6,804.9,1573.7,2038.1,2303.3,2681.0,2870.5,3838.8,4928.9,6073.4]; % Hz
omega_n_y = freq_n_y.*2*pi();
% Hz → rad/s, nat freq for mode y (Normal Direction)
zeta_y = [0.021, 0.089,0.027,0.075,0.027,0.016,0.220,0.019,0.014,0.006,0.017,0.016];
sig_y = [ 9.797268*10^-1,13.49004,-31.06831, 30.33849, 1.283945,1.298625, 0.9518897,11.90834,-19.12932,-11.19589,-18.59880,-7.608392].*10^-6;
nu_y = [5.959362*10^-2, 4.544067*10^-1,3.816475,8.813244,8.678961*10^-1,1.270846,3.750897,2.825781, 4.051860, 8.475725,8.993226, 2.022991].*10^-5;

% Answer
% Step 1. Calculating the transfer functions
% Dampened Natural Frequencies %
omega_d_x = omega_n_x.*sqrt(1-zeta_x); %rad/s, Dampened nat freq
omega_d_y = omega_n_y.*sqrt(1-zeta_y); %rad/s, Dampened nat freq

% Residuals %
alpha_x = 2.*(zeta_x.*omega_n_x.*sig_x - omega_d_x.*nu_x);
alpha_y = 2.*(zeta_y.*omega_n_y.*sig_y - omega_d_y.*nu_y);
beta_x = 2.*sig_x;
beta_y = 2.*sig_y;

% Transfer Functions %
transfer_func_Hxx = ...
    tf([beta_x(1),alpha_x(1)],[1,2*zeta_x(1)*omega_n_x(1),omega_n_x(1)^2]) + ...
    tf([beta_x(2),alpha_x(2)],[1,2*zeta_x(2)*omega_n_x(2),omega_n_x(2)^2]) + ...
    tf([beta_x(3),alpha_x(3)],[1,2*zeta_x(3)*omega_n_x(3),omega_n_x(3)^2]) + ...
    tf([beta_x(4),alpha_x(4)],[1,2*zeta_x(4)*omega_n_x(4),omega_n_x(4)^2]) + ...
    tf([beta_x(5),alpha_x(5)],[1,2*zeta_x(5)*omega_n_x(5),omega_n_x(5)^2]) + ...
    tf([beta_x(6),alpha_x(6)],[1,2*zeta_x(6)*omega_n_x(6),omega_n_x(6)^2]) + ...
    tf([beta_x(7),alpha_x(7)],[1,2*zeta_x(7)*omega_n_x(7),omega_n_x(7)^2]) + ...
    tf([beta_x(8),alpha_x(8)],[1,2*zeta_x(8)*omega_n_x(8),omega_n_x(8)^2]) + ...
    tf([beta_x(9),alpha_x(9)],[1,2*zeta_x(9)*omega_n_x(9),omega_n_x(9)^2]);

transfer_func_Hyy = ...
    tf([beta_y(1),alpha_y(1)],[1,2*zeta_y(1)*omega_n_y(1),omega_n_y(1)^2]) +
    ...
    tf([beta_y(2),alpha_y(2)],[1,2*zeta_y(2)*omega_n_y(2),omega_n_y(2)^2]) +
    ...
    tf([beta_y(3),alpha_y(3)],[1,2*zeta_y(3)*omega_n_y(3),omega_n_y(3)^2]) +
    ...

```

```

tf([beta_y(4),alpha_y(4)],[1,2*zeta_y(4)*omega_n_y(4),omega_n_y(4)^2]) +
...
tf([beta_y(5),alpha_y(5)],[1,2*zeta_y(5)*omega_n_y(5),omega_n_y(5)^2]) +
...
tf([beta_y(6),alpha_y(6)],[1,2*zeta_y(6)*omega_n_y(6),omega_n_y(6)^2]) +
...
tf([beta_y(7),alpha_y(7)],[1,2*zeta_y(7)*omega_n_y(7),omega_n_y(7)^2]) +
...
tf([beta_y(8),alpha_y(8)],[1,2*zeta_y(8)*omega_n_y(8),omega_n_y(8)^2]) +
...
tf([beta_y(9),alpha_y(9)],[1,2*zeta_y(9)*omega_n_y(9),omega_n_y(9)^2]) +
...
tf([beta_y(10),alpha_y(10)],[1,2*zeta_y(10)*omega_n_y(10),omega_n_y(10)^2]) +
...
tf([beta_y(11),alpha_y(11)],[1,2*zeta_y(11)*omega_n_y(11),omega_n_y(11)^2]) +
...
tf([beta_y(12),alpha_y(12)],[1,2*zeta_y(12)*omega_n_y(12),omega_n_y(12)^2]);

% ==== %
% H_xx %
% ==== %
% H_xx: Transfer Function Plots: Magnitude & Phase %
f=0:.1:7000*2*pi(); %Matching the book's example (but in rad/s)
[magH phaH]=bode(transfer_func_Hxx,f);
magH = squeeze(magH);
phaH = squeeze(phaH);

subplot_tight(3,3,0+1,0.075); plot(f, magH, 'k');
title("H_{xx}: Magnitude [m/N] x Frequency [rad/sec]"); xlabel('Frequency
[rad/sec]'); ylabel('Magnitude [m/N]'); grid on;

subplot_tight(3,3,3+1,0.075); plot(f, phaH, 'k');
title("H_{xx}: Phase (\phi) [degree] x Frequency [rad/sec]"); xlabel('Frequency
[rad/sec]'); ylabel('Phase [degree]'); grid on;

% Hxx: Transfer Function Plots: Real & Imaginary %
[Real Imag]=nyquist(transfer_func_Hxx,f);
Real = squeeze(Real);
Imag = squeeze(Imag);

subplot_tight(3,3,0+2,0.075); plot(f, Real, 'k');
title('H_{xx}: Real Part [m/N] x Frequency [rad/sec]'); xlabel('Frequency
[rad/sec]'); ylabel('Magnitude - Real [m/N]'); grid on;

subplot_tight(3,3,3+2,0.075); plot(f, Imag, 'k');
title('H_{xx}: Imaginary Part [m/N] x Frequency [rad/sec]'); xlabel('Frequency
[rad/sec]'); ylabel('Magnitude - Imaginary [m/N]'); grid on;

my_export("A4_Q2_Transfer Functions(xx).png");

% ==== %
% H_yy %
% ==== %
% Hyy: Transfer Function Plots: Magnitude & Phase %

```

```

[magH phaH]=bode(transfer_func_Hyy,f);
magH = squeeze(magH);
phaH = squeeze(phaH);

figure;
subplot_tight(3,3,0+1,0.075); plot(f, magH, 'k');
title("H_{yy}: Magnitude [m/N] x Frequency [rad/sec]"); xlabel('Frequency
[rad/sec]'); ylabel('Magnitude [m/N]'); grid on;

subplot_tight(3,3,3+1,0.075); plot(f, phaH, 'k');
title("H_{yy}: Phase (\phi) [degree] x Frequency [rad/sec]"); xlabel('Frequency
[rad/sec]'); ylabel('Phase [degree]'); grid on;

% Hyy: Transfer Function Plots: Real & Imaginary %
[Real Imag]=nyquist(transfer_func_Hyy,f);
Real = squeeze(Real);
Imag = squeeze(Imag);

subplot_tight(3,3,0+2,0.075); plot(f, Real, 'k');
title('H_{yy}: Real Part [m/N] x Frequency [rad/sec]'); xlabel('Frequency
[rad/sec]'); ylabel('Magnitude - Real [m/N]'); grid on;

subplot_tight(3,3,3+2,0.075); plot(f, Imag, 'k');
title('H_{yy}: Imaginary Part [m/N] x Frequency [rad/sec]'); xlabel('Frequency
[rad/sec]'); ylabel('Magnitude - Imaginary [m/N]'); grid on;

my_export("A4_Q2_Transfer Functions(yy).png");

% Step 2. Finding Eigenvalues of the system
alpha_xx = 0.5*(+cos(2*phi_ex) - 2*K_r*phi_ex + K_r*sin(2*phi_ex)) -
0.5*(+cos(2*phi_st) - 2*K_r*phi_st + K_r*sin(2*phi_st));
alpha_xy = 0.5*(-sin(2*phi_ex) - 2*phi_ex + K_r*cos(2*phi_ex)) - 0.5*(-
sin(2*phi_st) - 2*phi_st + K_r*cos(2*phi_st));
alpha_yx = 0.5*(-sin(2*phi_ex) + 2*phi_ex + K_r*cos(2*phi_ex)) - 0.5*(-
sin(2*phi_st) + 2*phi_st + K_r*cos(2*phi_st));
alpha_yy = 0.5*(-cos(2*phi_ex) - 2*K_r*phi_ex - K_r*sin(2*phi_ex)) - 0.5*(-
cos(2*phi_st) - 2*K_r*phi_st - K_r*sin(2*phi_st));

% Selecting chatter frequencies + Scan Windows
freq1      = 600; % Hz
freq2      = 3600; % Hz
freq3      = 4600; % Hz
% scan_window = 600; % Hz
scan_window = 300; % Hz
resolution = 0.5; % Hz

Testfreq1 = freq1:0.1:freq1+scan_window;
Testfreq2 = freq2:0.1:freq2+scan_window;
Testfreq3 = freq3:0.1:freq3+scan_window;
Testfreq = [Testfreq1,Testfreq2,Testfreq3];
test_length = length(Testfreq);

for j = 1:test_length
    freq_c = Testfreq(j);

```

```

omega_c = freq_c*2*pi();
[Gx,Hx]=nyquist(transfer_func_Hxx,omega_c);
[Gy,Hy]=nyquist(transfer_func_Hyy,omega_c);
Ixx = Gx + Hx*1i; Iyy = Gy + Hy*1i;
a0 = Ixx*Iyy*(alpha_xx*alpha_yy-alpha_xy*alpha_yx);
a1 = alpha_xx*Ixx + alpha_yy*Iyy;
lambda = -1/(2*a0)*(a1-sqrt((a1^2)-4*a0));
kappa = imag(lambda)/real(lambda);
psi = atan2(imag(lambda),real(lambda));
a_lim = -2*pi()*real(lambda)/(N*K_t)*(1+kappa^2);

for k = 1:20 % Stability Lobes
    T = 1/omega_c*((pi()-2*psi)+2*(k)*pi());
    stability_matrix((1+20*(j-1)+k-1),1) = a_lim;
    stability_matrix((1+20*(j-1)+k-1),2) = 60/(N*T);
end

end

stability_matrix = sortrows(stability_matrix,2);
stability_matrix_size = size(stability_matrix, 1);
resolution = 80;
lim = round(stability_matrix_size/resolution);

for i = 1:lim
    if i*resolution < stability_matrix_size
        tmp = stability_matrix(((i-1)*resolution+1):i*resolution,:);
        [M,index] = min(tmp(:,1));
        stability_lobes_data(i,:) = tmp(index,:);
    end
end

% Plotting of Stability %

figure;
x = stability_lobes_data(:,2);
y = stability_lobes_data(:,1);
subplot_tight(3,3,0+1,0.075); plot(x,y, 'k', 'linewidth', 1.0)

xlim([0 20000]); % x_max = same as book
ylim([0 12]);
%ylim([0 40]);
xticks(0:2000:20000);
grid on;
title('Stability Lobes - Slotting');
%title('Stability Lobes - Half Immersion Down Milling');
xlabel('Spindle Speed [rev/min]'); ylabel('Depth of Cut, a_{lim} [mm]');
my_export("A4_Q2_Stability_Lobes_Slotting");
%my_export("A4_Q2_Stability_Lobes_Half_Immersion");

function y = my_export(title)
    s = get(0, 'ScreenSize');
    set(gcf, 'Position', [0 0 s(3) s(4)]);
    set(gcf, 'Color', 'w');
    export_fig(title, "-png", "-r300");

```

end